

Cifra de Vigenère: Implementação e Criptoanálise

Trabalho de Implementação I — CIC0201 Segurança Computacional

Ana Luísa Reis Nascente
Matrícula: 211045688

Gabriel de Sousa
Matrícula: 211056000

Marina Pimentel Moreno
Matrícula: 222014071

Pedro de Paula Campos
Matrícula: 231036050

Resumo—Este trabalho apresenta a implementação da cifra de Vigenère e de um ataque para recuperação da chave por propriedades estatísticas da linguagem. A implementação inclui cifração, decifração, estimativa do tamanho por Índice de Coincidência e recuperação da chave por análise de frequência. Além da validação funcional, 7.200 ataques controlados quantificaram o efeito do tamanho da mensagem, da chave e do embaralhamento. Para uma chave de nove letras, o pipeline recuperou a chave exata em 96,7% das tentativas com 432 letras. Benchmarks também avaliaram vazão, tamanho das palavras e paralelismo em lotes.

Index Terms—cifra de Vigenère, criptografia clássica, criptoanálise, índice de coincidência, análise de frequência

I. INTRODUÇÃO

Este relatório se insere no módulo de Criptografia Simétrica da disciplina CIC0201 – Segurança Computacional, e documenta o Trabalho de Implementação 1: a cifra de Vigenère, cifra de substituição polialfabética historicamente relevante por ilustrar, de forma simples, dois princípios centrais da criptografia moderna — a diferença entre um espaço de chaves grande e segurança real, e como a reutilização cíclica de uma chave curta introduz uma estrutura estatística explorável [1].

O trabalho está dividido em duas partes complementares: (i) um cifrador/decifrador implementado do zero, sem bibliotecas criptográficas prontas; e (ii) um ataque de criptoanálise capaz de recuperar a chave de um criptograma desconhecido a partir de propriedades estatísticas da língua portuguesa e inglesa. O restante deste relatório está organizado da seguinte forma: a Seção II apresenta a fundamentação teórica da cifra e as decisões de implementação da Parte I; a Seção III descreve o ambiente experimental, a metodologia do ataque da Parte II e os resultados intermediários; a Seção IV apresenta as conclusões técnicas; e a bibliografia lista as referências utilizadas.

II. FUNDAMENTAÇÃO TEÓRICA

A. A cifra de Vigenère

A cifra de Vigenère desloca cada letra do texto claro por um valor que depende da posição correspondente de uma chave, repetida ciclicamente sobre a mensagem. Para uma letra do texto claro P_i e a letra da chave na posição $i \bmod L$ (L = tamanho da chave), convertidas para valores numéricos de 0 a 25:

$$\text{Cifração: } C_i = (P_i + K_{i \bmod L}) \bmod 26 \quad (1)$$

$$\text{Decifração: } P_i = (C_i - K_{i \bmod L} + 26) \bmod 26 \quad (2)$$

Diferentemente da cifra de César (deslocamento único), a chave do Vigenère é uma sequência, gerando um espaço de chaves de tamanho 26^L — para $L = 14$, por exemplo, aproximadamente 2^{66} chaves possíveis, tornando a busca exaustiva inviável. Essa aparente robustez não impede, contudo, o ataque estatístico descrito na Seção III, precisamente porque a chave se repete ciclicamente [2].

B. Tratamento do alfabeto

O grupo adotou a seguinte política, comum a toda a implementação:

- Alfabeto de cifração: A–Z (ASCII), sem distinção entre maiúsculas e minúsculas para o cálculo do deslocamento.
- O caso original (maiúscula/minúscula) de cada letra é preservado no criptograma.
- Espaços, números, pontuação e demais caracteres fora de A–Z são copiados sem alteração e **não consomem posição da chave**.
- A chave é validada: deve ser não vazia e conter apenas letras A–Z.

C. Fundamentos da criptoanálise (Índice de Coincidência)

O ataque de recuperação de chave apoia-se no Índice de Coincidência (IC), medida proposta por Friedman para quantificar a concentração estatística das letras [3]:

$$IC = \frac{\sum_i n_i(n_i - 1)}{N(N - 1)} \quad (3)$$

onde n_i é a frequência da letra i no texto e N o tamanho do texto. A distribuição não uniforme do português usada no projeto tem IC teórico $\sum_i p_i^2 \approx 0,078$; uma distribuição uniforme (como um criptograma de Vigenère mal alinhado) tem IC próximo de $1/26 \approx 0,038$. Essa diferença é a base para estimar o tamanho da chave (detalhado na Seção III).

III. AMBIENTE EXPERIMENTAL E ANÁLISE DE RESULTADOS

A. Descrição do Cenário

- **Linguagem e ferramentas:** C++17, biblioteca padrão, CMake 3.28.3 e testes executáveis próprios, sem bibliotecas de cifra ou criptoanálise pronta. Os gráficos foram renderizados com Matplotlib 3.10.7 a partir dos CSVs preservados no projeto.
- **Ambiente de compilação/execução:** Ubuntu 24.04.4, g++ 13.3.0 com `-O3 -DNDEBUG`, em um AMD Ryzen

7 5700U (8 núcleos físicos, 16 processadores lógicos) e 5,6 GiB de RAM visíveis ao sistema.

- **Dados experimentais:** criptograma de demonstração em português, produzido pelo grupo a partir de um texto claro conhecido e da chave `SEGURANCA`. Os arquivos de entrada e referência estão em `textos para testes/`. Como os criptogramas previstos no roteiro não foram fornecidos, a professora autorizou o uso de dados de teste produzidos pelo grupo.
- **Estrutura do código:** biblioteca de ciframento reutilizável (`parte1_cifrador/`), acompanhada de interface de linha de comando, testes automatizados e benchmark de desempenho; módulo de criptoanálise (`parte2_criptoanalise/`) com estimador de tamanho, análise de frequência e pipeline para recuperação da chave e do texto claro.

B. Metodologia da Criptoanálise

Metodologia do ataque de recuperação de chave, alinhada à fundamentação da Seção II:

- 1) **Estimativa do tamanho da chave:** para cada tamanho candidato k , o criptograma é dividido em k subconjuntos (posições $0, k, 2k, \dots; 1, k+1, 2k+1, \dots$; etc.) e o IC médio dos subconjuntos válidos é calculado. Os tamanhos são ordenados pelo maior IC médio; quando a diferença é inferior à tolerância de 0,003, o menor tamanho é priorizado, reduzindo falsos candidatos que sejam múltiplos da chave real.
- 2) **Recuperação de cada letra da chave:** com o tamanho k estimado, cada subconjunto corresponde a um deslocamento fixo (equivalente a uma Cifra de César). Calcula-se a frequência relativa de cada letra do subconjunto e testam-se os 26 deslocamentos possíveis, comparando a distribuição observada com a distribuição esperada do idioma [4] pela estatística de qui-quadrado; o deslocamento de menor valor é adotado como candidato.
- 3) **Reconstrução e validação:** as letras recuperadas formam a chave candidata, usada para decifrar o criptograma completo. O executável apresenta vários tamanhos candidatos e seus textos claros para avaliação de legibilidade; se necessário, letras específicas são refinadas manualmente.

C. Protocolo dos Experimentos

Para não confundir interceptação da comunicação com sucesso do ataque, definiu-se *interceptação bem-sucedida* como a recuperação da chave exata no primeiro candidato do pipeline. Foram testadas chaves aleatórias de 3, 6, 9 e 12 letras, com 4 a 128 letras de criptograma por posição da chave (N/L). Para cada combinação foram feitas 60 tentativas determinísticas, com o estimador procurando períodos de 1 a $2L$. As amostras vieram do texto de demonstração, variando o ponto inicial, sob três condições: ordem original, palavras embaralhadas e letras embaralhadas. Assim, a taxa apresentada é empírica para este corpus e este ataque, não uma probabilidade universal.

Nos ensaios de tempo, cada ponto é a mediana de cinco rodadas. A vazão da cifra processou ao menos 64 MiB por rodada; no teste paralelo, cada thread processou uma mensagem independente de 8 MiB seis vezes. Essa separação é necessária porque o núcleo implementado cifra uma única mensagem em uma thread; o paralelismo mede capacidade de atender um lote, sem atribuir ao código uma paralelização que ele não possui.

D. Análise de Resultados

Os componentes foram validados separadamente e em conjunto. O cifrador/decifrador passou pelos testes existentes, incluindo o vetor clássico, preservação de caixa, caracteres não alfabéticos, chave inválida, CLI de decifração e 500 casos determinísticos de ida e volta. O estimador identificou como primeira opção os tamanhos corretos 5, 7, 11 e 16 nos quatro criptogramas de teste; os testes de análise de frequência também passaram. Por fim, um teste integrado recuperou tamanho, chave e texto claro conhecidos. No total, os sete testes registrados no CMake raiz foram aprovados.

No experimento de demonstração, o programa recebeu apenas o idioma português, o limite de tamanho 20 e o criptograma. O primeiro resultado foi o tamanho 9 e a chave `SEGURANCA`; a decifração coincidiu integralmente com o texto de referência, preservando espaços, pontuação e parágrafos. O segundo resultado foi o tamanho 18 e a chave `SEGURANCASEGURANCA`, que gera o mesmo texto claro por ser uma repetição da chave fundamental.

1) *Letras mais usadas e estrutura que sobrevive à cifra:* No corpus observado, as letras mais frequentes foram A (15,81%), E (12,86%) e O (9,27%), próximas das três maiores frequências da tabela de português usada pelo ataque. A Fig. 1(a) mostra que a chave de nove letras espalha esses picos por diferentes deslocamentos: a distribuição marginal do criptograma fica mais plana, mas não uniforme. O ataque não compara o criptograma inteiro diretamente com o idioma; ele separa as posições cifradas pela mesma letra da chave, desfazendo a mistura.

Essa separação fica visível na Fig. 1(b): o IC médio sobe de aproximadamente 0,042 nos candidatos mal alinhados para 0,0775 em $L = 9$ e 0,0762 em $L = 18$. Tamanhos múltiplos podem apresentar IC elevado; por isso, o estimador usa uma tolerância e favorece o menor candidato quando os valores são próximos. O resultado de tamanho 18 confirma que uma repetição da chave fundamental também decifra corretamente.

2) *Quando o ataque passa a funcionar:* A Fig. 2 mostra que o parâmetro decisivo não é só o tamanho absoluto da mensagem ou da chave, mas quantas observações cada posição da chave recebe, N/L . Se L já era conhecido, todos os tamanhos testados alcançaram ao menos 88,3% de recuperação exata com 24 letras por posição e 100% com 48. No pipeline completo, que também precisa descobrir L , a transição foi mais lenta: para $L = 9$, o sucesso passou de 30% em $N/L = 16$ (144 letras) para 96,7% em $N/L = 48$ (432 letras). Todas as quatro chaves atingiram 100% em $N/L = 128$ no corpus ensaiado.

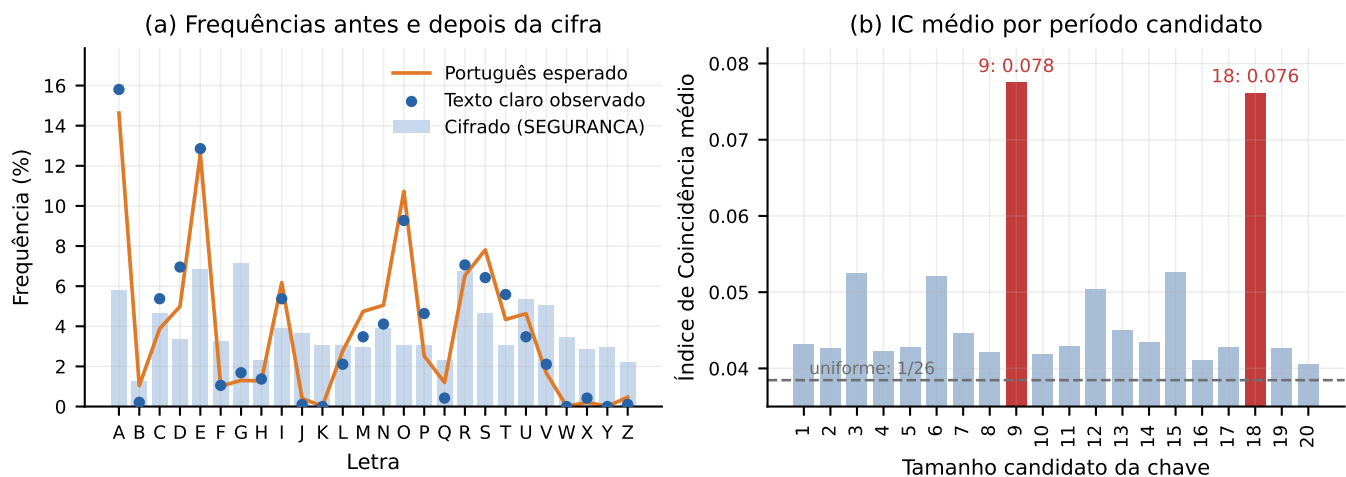


Figura 1. Relação entre frequência e periodicidade. (a) Frequências de letras esperadas em português, observadas no texto claro e após cifração com SEGURANCA. (b) IC médio do criptograma para períodos de 1 a 20; os picos em 9 e 18 revelam o período fundamental e seu múltiplo.

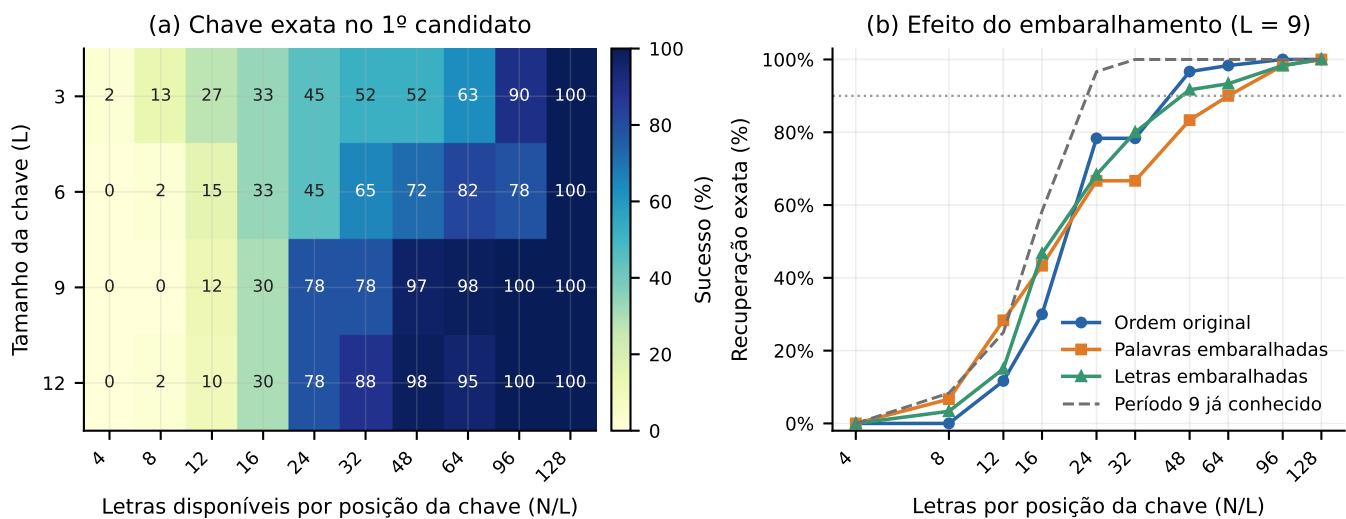


Figura 2. Taxa empírica de recuperação da chave exata em 60 tentativas por ponto. (a) Efeito conjunto de L e N/L no pipeline completo, para o texto em ordem original. (b) Ordem original, palavras embaralhadas e letras embaralhadas para $L = 9$; a curva tracejada isola a análise de frequência quando o período correto já é conhecido.

Embaralhar não constitui defesa contra este método, pois preserva as frequências de uma letra e não remove a repetição da chave. Para $L = 9$ e 864 letras ($N/L = 96$), a recuperação ficou em 100% na ordem original e 98,3% nos dois tipos de embaralhamento. As diferenças nos pontos intermediários decorrem de quais letras caem em cada coluna e da amostra de 60 tentativas, mas as três curvas convergem quando há texto suficiente.

O comprimento das palavras do texto claro também não altera a segurança se a sequência de letras for mantida: espaços são preservados e não avançam a chave. Em contraste, uma palavra-chave maior divide o mesmo texto em mais colunas e reduz a amostra de cada coluna; por isso mensagens maiores são necessárias para manter o mesmo N/L .

3) *Eficiência computacional e uso de núcleos*: A Fig. 3 reúne os resultados computacionais. O núcleo *in-place* sustentou cerca de 771–777 MiB/s entre 1 KiB e 1 MiB e caiu para 498 MiB/s em 64 MiB, região em que cópias e hierarquia de memória passam a dominar. A API com retorno é ligeiramente mais lenta por copiar a entrada; em 4 MiB, por exemplo, foram 650 contra 740 MiB/s.

Palavras curtas parecem mais rápidas por byte: com uma letra entre espaços, só 50% dos bytes exigem deslocamento e a vazão foi 1027 MiB/s; sem espaços, todos os bytes são cifrados e a vazão foi 690 MiB/s. Isso não é ganho criptográfico: normalizar pelo número de letras úteis elimina essa vantagem. O custo do ataque cresceu aproximadamente de forma linear após o custo fixo inicial, chegando a 1,58 ms para 16 mil letras e cinco candidatos.

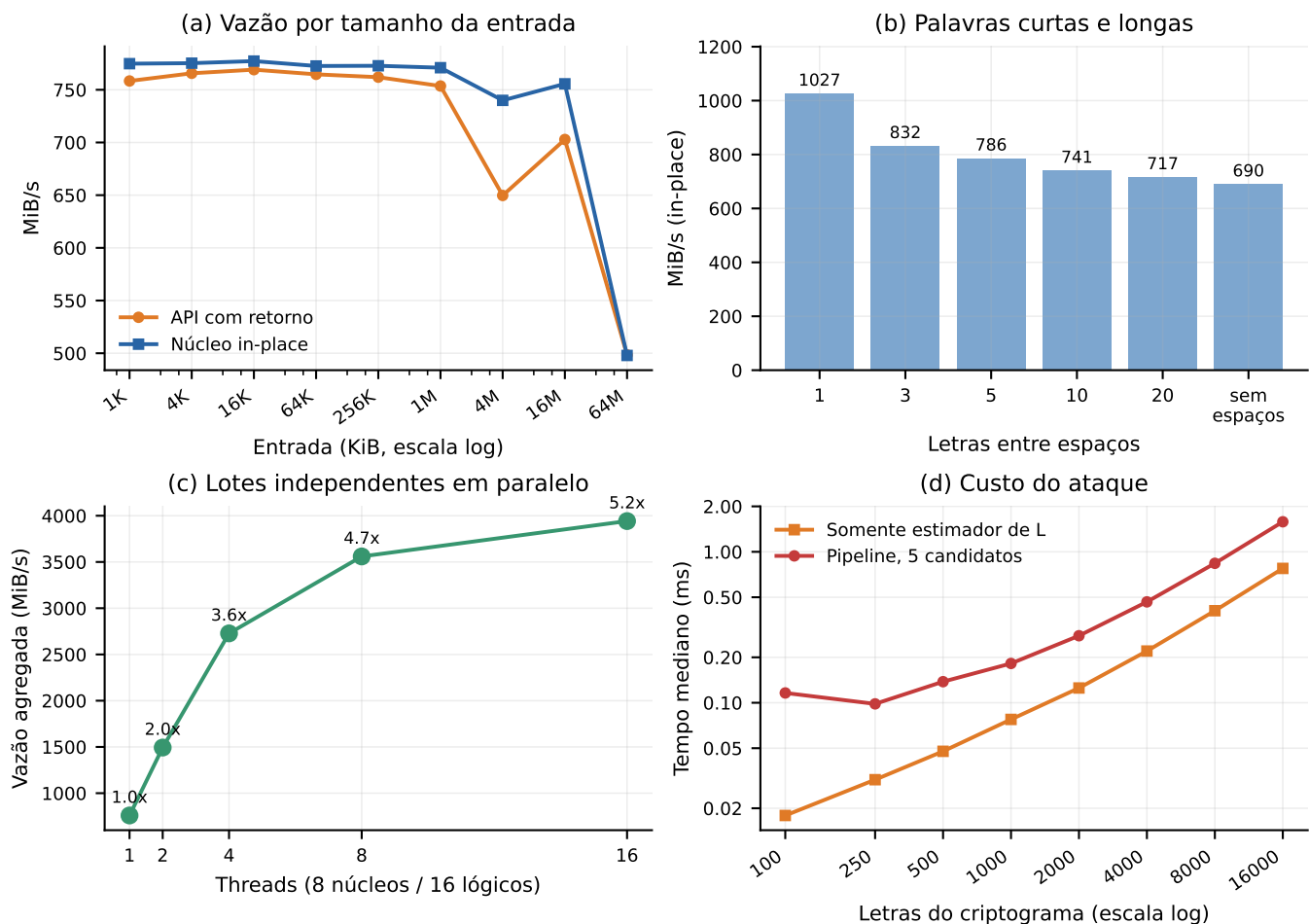


Figura 3. Desempenho medido no Ryzen 7 5700U. (a) Vazão por tamanho e API. (b) Vazão por byte para entradas de 16 MiB com diferentes quantidades de letras entre espaços. (c) Vazão agregada ao cifrar mensagens independentes; uma única mensagem contínua monothread. (d) Tempo do estimador e do pipeline com limite de chave 20 e cinco candidatos.

Em lotes, a vazão aumentou de 759 MiB/s com uma thread para 3560 MiB/s com oito e 3942 MiB/s com 16, respectivamente 4,69 e 5,19 vezes a base. O ganho pequeno de 8 para 16 threads (10,7%) indica saturação após os oito núcleos físicos, por SMT e largura de banda de memória. Portanto, a forma útil de explorar vários núcleos nesta versão é cifrar mensagens independentes; uma única mensagem não fica mais rápida automaticamente.

IV. CONCLUSÕES

Os resultados demonstram que 26^L chaves possíveis não garantem segurança quando a chave se repete. Com o período conhecido, 24 letras por posição deram ao menos 88,3% de sucesso; no pipeline completo com $L = 9$, 432 letras deram 96,7%. Embaralhar palavras ou letras não removeu a vulnerabilidade, pois preserva frequências e periodicidade.

Computacionalmente, o núcleo atingiu 771–777 MiB/s até 1 MiB, e lotes independentes chegaram a 5,19 vezes a vazão de uma thread; uma mensagem isolada permanece monothread. As taxas são limitadas ao corpus e à máquina ensaiados, mas expõem a variável central: a quantidade de amostras por

posição da chave. Uma chave aleatória, do tamanho do texto e usada uma única vez (*One-Time Pad*) eliminaria essa repetição e tornaria o ataque estatístico ineficaz.

REFERÊNCIAS

- [1] Roteiro do Trabalho de Implementação 1 — CIC0201, Profa. Priscila Solis, UnB, 2026.
- [2] Slides de Criptografia Fundamentos (*lec1-2026, lec2-total, introduction-part2*) — CIC0201, UnB, 2026.
- [3] W. F. Friedman, *The Index of Coincidence and Its Applications in Cryptography*. Riverbank Laboratories, 1922.
- [4] Wikipédia, “Frequência de letras.” [Online]. Disponível em: https://pt.wikipedia.org/wiki/Frequencia_de_letras. Acesso em: 31 ago. 2026.

Repositório do projeto: <https://github.com/anaareis/vigenere-seg-comp>